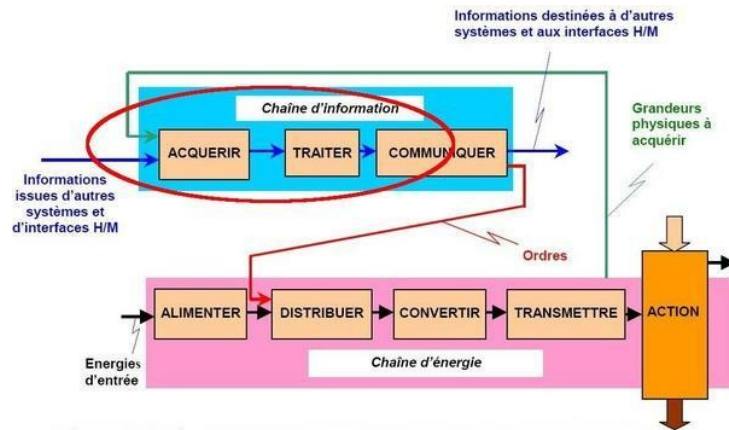


1 Présentation

1.1 Constat Tous les systèmes présents dans le laboratoire sont pilotés par une unité centrale électronique bâtie autour d'un microcontrôleur. Tous les produits qui nous entourent sont généralement pilotés par un microprocesseur ou un microcontrôleur voire par un ou plusieurs processeurs comme sur les pc.

1.2 Chaîne d'information



2 Numérisation d'un signal : fréquence d'échantillonnage et résolution

Avantages : reproduction à l'identique, la correction des erreurs, la transmission, l'insensibilité aux parasites...

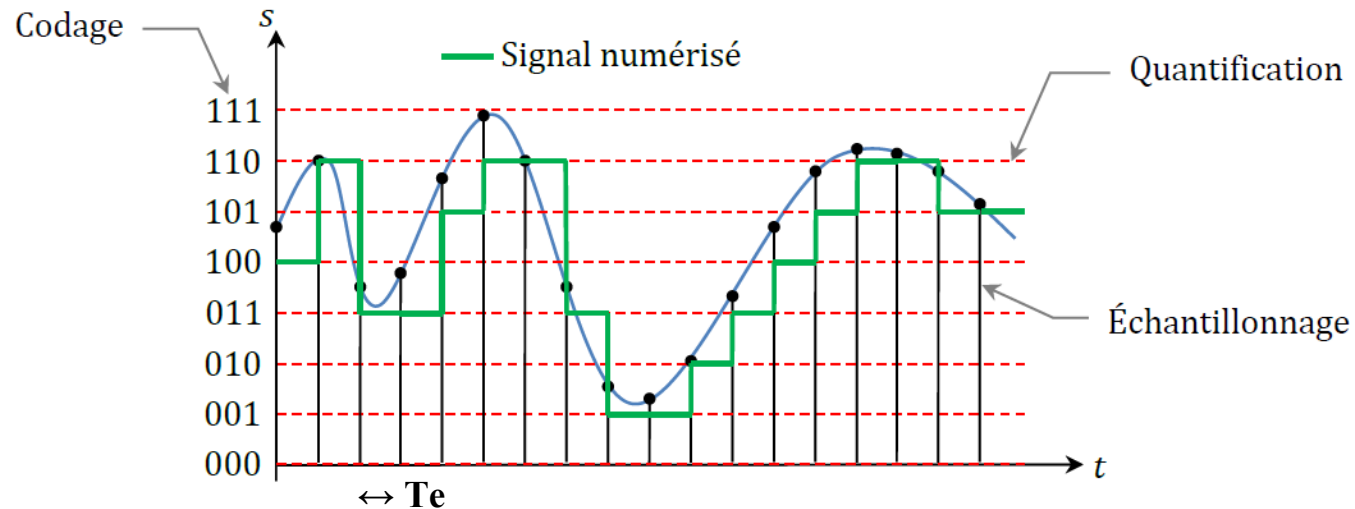
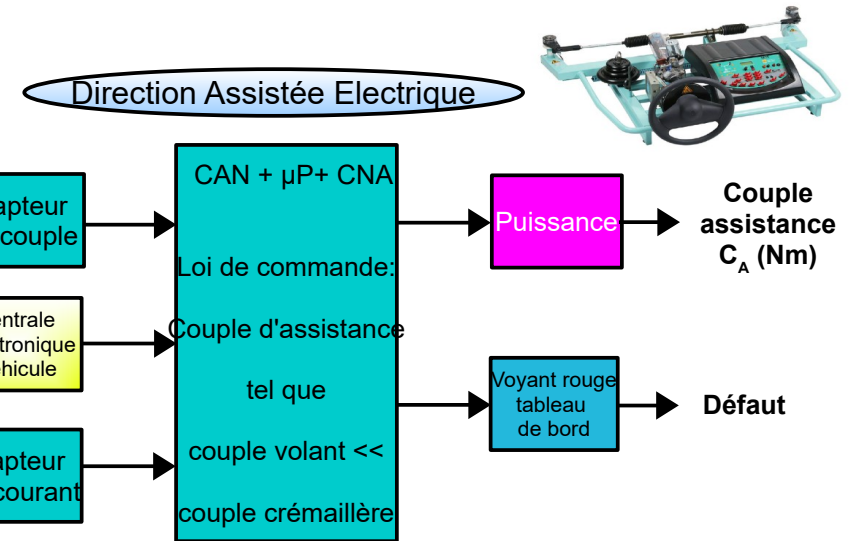
Inconvénients : perte relative d'information

Déf : Le signal bleu est qualifié d' **ANALOGIQUE** ou **CONTINU** au sens mathématique.

Déf : Le signal vert est qualifié de **NUMÉRIQUE** ou **DISCONTINU** ou **DISCRÉTISÉ**.

Déf : La **période d'échantillonnage** est définie par : $T_e = 1 / F_e$.

Déf : La **résolution** est le nombre de bits codés : ici 3 bits donc 2^3 valeurs possibles (de 000 à 111).



Temps	0	T_e	$2T_e$	$3T_e$	$4T_e$	$5T_e$	$6T_e$	$7T_e$	$8T_e$	$9T_e$	$10T_e$	$11T_e$	$12T_e$
$S_{(2)}$	100	110	011	011	101	110	110	011	001	001			

Principe de l'échantillonnage d'un point de vue mathématique : c'est le produit du signal d'entrée $S_a(t)$ par une fonction $x(t)$ qui vaut 1 aux instants $n.T_e$ sinon zéro.

La numérisation se résume donc en deux termes : la **fréquence d'échantillonnage** $F_{\text{éch}}$ en Hz ou en Msps pour Méga sample per second ou Million d'échantillons par seconde et la **résolution en nombre de bits**.

La conséquence de la combinaison de ces deux paramètres est la taille du fichier lorsque le signal numérisé est enregistré. Calculons par exemple la taille M d'un fichier contenant une minute de son en stéréo en Mio.

$$\text{Taille (bits)} = \text{durée en seconde} \times F_{\text{éch}} \text{ (Hz)} \times \text{résolution en bits} \times \text{nombre de canaux}$$

$$M = 60.44100.16.2 = 84672000 \text{ bits soit } 10584000 \text{ octets soit } 10335 \text{ Kio soit environ } 10\text{Mio} !!$$

3 Théorème de Shannon

Une fréquence très élevée n'apporte pas plus d'informations mais affecte la taille du fichier ou de la mémoire. Une fréquence d'échantillonnage trop faible pose un autre problème.

Constat : A droite, le signal numérisé a une période de 40ms soit une fréquence de 25Hz soit 1000Hz – 975Hz !

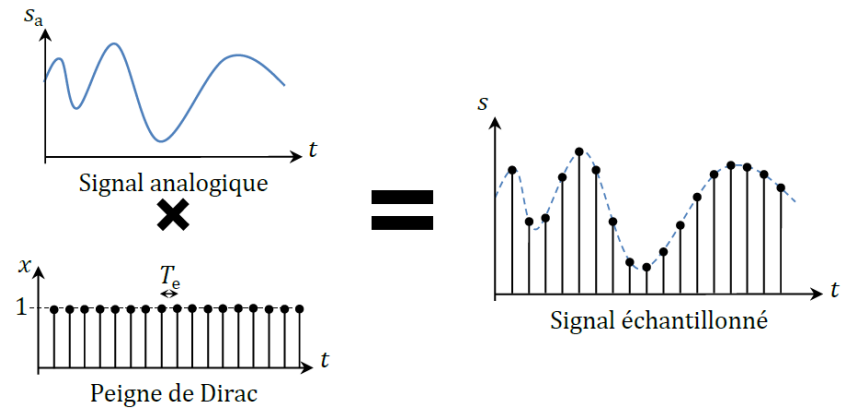
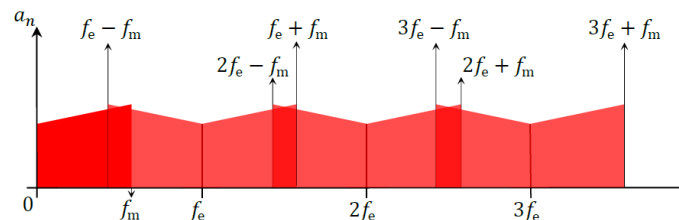
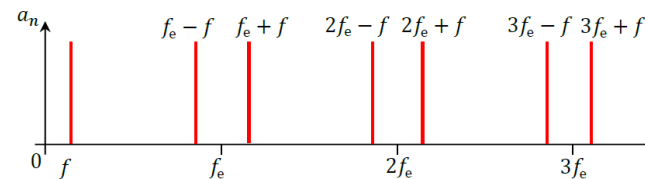
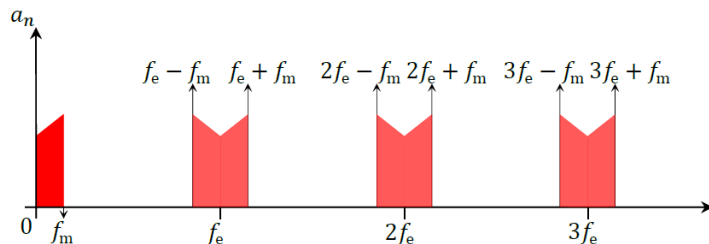
Principe physique : C'est le phénomène de **repliement de spectre** décrit par Shannon.

Justification du repliement du spectre :

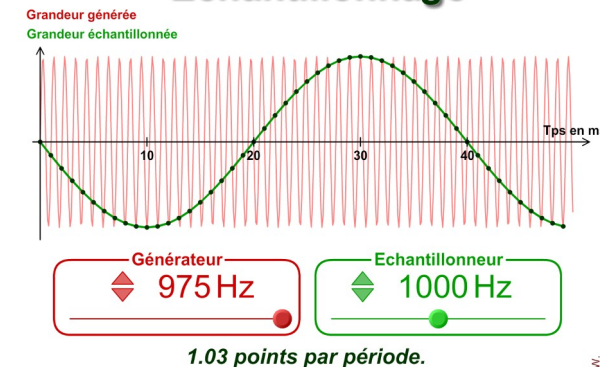
Spectre d'un signal sinusoïdal pur de fréquence f échantillonné à f_e :

Spectre d'un signal contenant des fréquences inférieures à f_m tel que $f_m < f_e$.

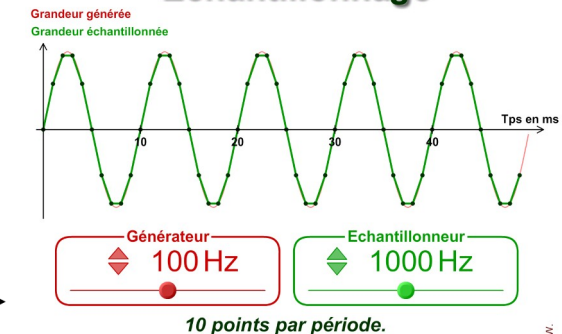
Spectre d'un signal contenant des fréquences inférieures à f_m tel que $f_m > f_e/2$. L'échantillonnage déforme le signal d'origine.



Echantillonnage



Echantillonnage



Théorème de Shannon ou critère de Nyquist-Shannon : l'échantillonnage d'un signal de fréquence maximale f_m nécessite une fréquence f_e telle que $f_e > 2.f_m$.

4 Conversion Analogique/Numérique

Déf : Un **Convertisseur Analogique / Numérique** est un circuit qui quantifie une tension d'entrée $V_e(t)$ en une valeur numérique N codée en binaire naturel sur n bits par rapport à une tension de référence V_{ref} .

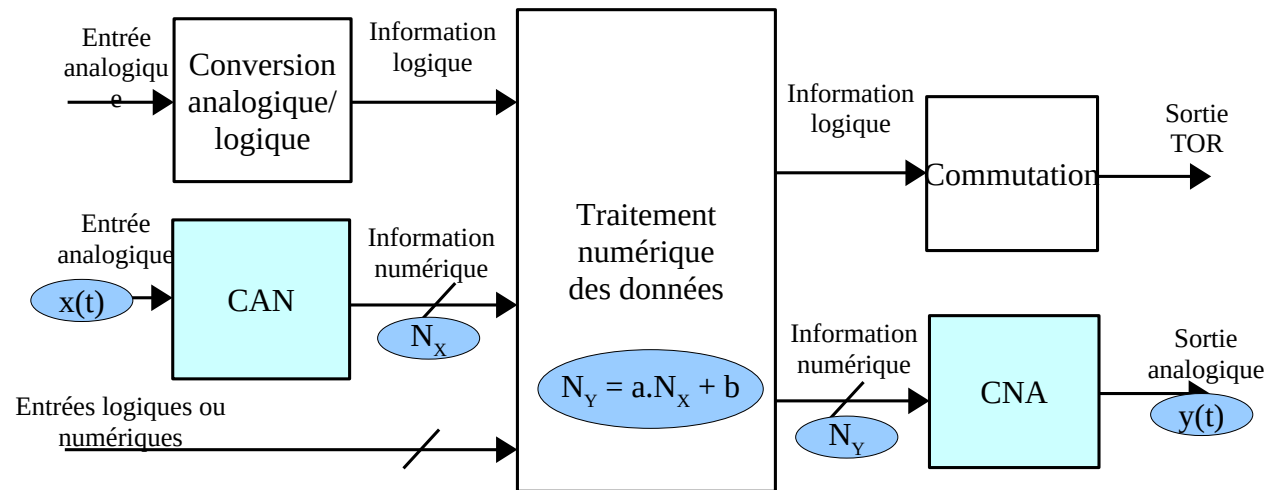
CAN = ADC

Convertisseur Analogique / Numérique
Analog to Digital Converter

Relations

$$N_{(10)} = PE \left(2^n \cdot \frac{V_e}{V_{ref}} \right)$$

$$N_{(10)} = 2^n \cdot \frac{V_e}{V_{ref}} = \frac{V_e}{q}$$



Déf : Le **quantum** q ou LSB est la plus petite tension « numérisable » c.à.d. lue par le CAN, $q = \frac{V_{ref}}{2^n}$ et est exprimé en Volt.

Déf : un **convertisseur unipolaire** a une plage de tension d'entrée strictement positive $0 < V_e < V_{ref}$.

Déf : un **convertisseur bipolaire** a une plage de tension d'entrée négative ou positive $-V_{ref} < V_e < +V_{ref}$. Dans ce cas :

$$N_{(10)} = PE \left(2^{(n-1)} + 2^{(n-1)} \cdot \frac{V_e}{V_{ref}} \right)$$

5 Conversion Numérique/Analogique

Déf : Un **Convertisseur Numérique / Analogique** est un circuit qui convertit une valeur numérique N codée en binaire naturel sur n bits en une tension de sortie V_s en fonction d'une référence fixe V_{ref} .

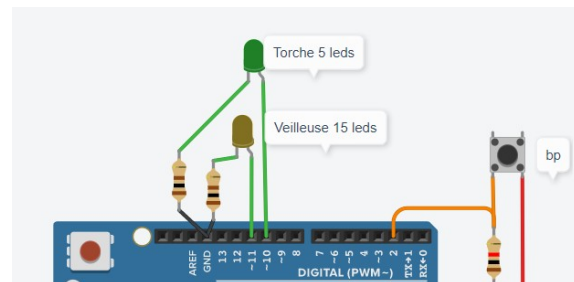
CNA = DAC

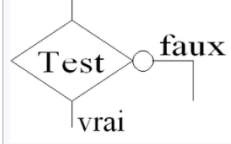
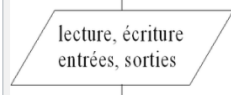
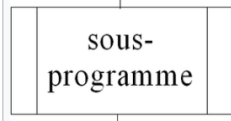
Convertisseur Numérique / Analogique
Digital to Analog Converter

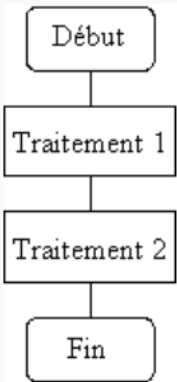
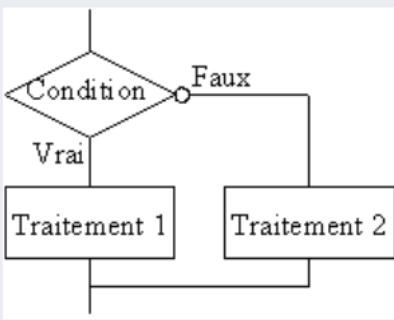
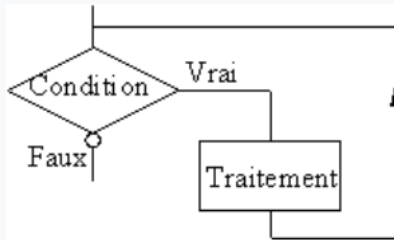
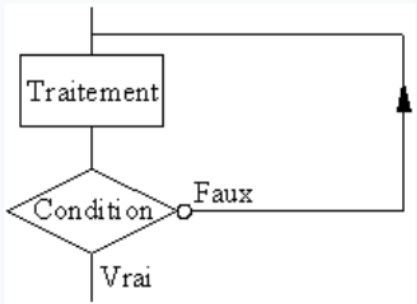
$$V_s = V_{ref} \cdot \frac{N}{2^n} = q \cdot N$$

6 Méthode de développement d'un programme

Affectation des entrées/sorties
Algorithme (texte) en pseudo-code ou algorithme (graphique)
Traduction en **Python**
Implantation dans la cible
Débogage



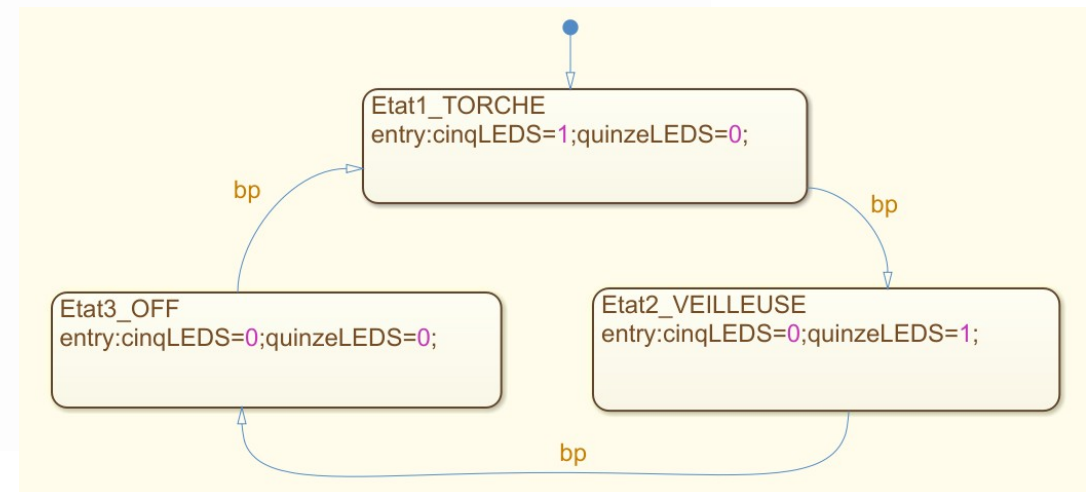
Symboles normalisés	Commentaires:
	Les tests ou branchements conditionnels : • l'entrée du test est l'angle du haut ; • la sortie avec le rond est le résultat du test lorsqu'il est faux ; • la sortie sans rond est le résultat du test lorsqu'il est vrai.
	Mise à disposition d'une information à traiter ou enregistrement d'une information traitée.
	Appel de sous-programme.

Séquence linéaire	Séquence alternative « si...alors...sinon »	Séquence répétitive « tant que...faire... »	Séquence répétitive « répéter...jusqu'à... »
			
Début • « Traitement 1 » • « Traitement 2 » Fin	Si « condition » • alors « Traitement 1 » • sinon « Traitement 2 » Fin si	Tant que « condition » • faire « traitement » Fin tant que	Répéter « traitement » jusqu'à « condition »

7 Traduction en algorithme puis en Python du graphe d'états

```
1 from machine import Pin
2 import time
3 # Lampe-torche à partir du diagramme d'états
4 led_torche = Pin(15, Pin.OUT) # La led torche est sur la broche de sortie n°15
5 led_veilleuse = Pin(13, Pin.OUT) # La led veilleuse est sur la broche de sortie n°13
6 button_BP = Pin(14, Pin.IN, Pin.PULL_DOWN) # Le BP est connecté sur l'entrée repérée 14
7
8 etat_torche=1
9 etat_veilleuse=0
10 etat_off=0
11 led_torche.on()
12
13 # La fonction button_BP() renvoie la valeur 1 si le bouton-poussoir est activé
14 while True:
15     if button_BP() and etat_torche==1:
16         time.sleep(0.50) # Attente 50ms pour anti-rebonds
17         while button_BP(): button_BP() # Attente bouton-poussoir relâché
18         etat_veilleuse=1 # Mise à jour états
19         etat_torche=0
20         led_veilleuse.on() # Mise à jour des sorties led
21         led_torche.off()
22     if button_BP() and etat_veilleuse==1:
23         time.sleep(0.50)
24         while button_BP(): button_BP()
25         etat_off=1
26         etat_veilleuse=0
27         led_veilleuse.off()
28         led_torche.off()
29     if button_BP() and etat_off==1:
30         time.sleep(0.50)
31         while button_BP(): button_BP()
32         etat_off=0
33         etat_torche=1
```

Début
Initialiser variables
Activer état_torche par défaut
Itérer toujours
Lire bp
Si bp=1 et état_torche=1
Alors désactiver état_torche
Activer état_veilleuse
Attendre bp relâché
Activer quinzeLEDS
Désactiver cinqLEDS
Fin du Si
Si



COMPÉTENCES SAVOIR-FAIRE

Connaissances : CAN, CNA, algorithme en pseudo-code, algorithme

Compétences évaluées :

- analyser et compléter un algorithme
- traduire un diagramme d'états en pseudocode
- justifier le choix d'un CAN (gamme d'entrée, résolution)